# Hierarchical and Mixed Models

## Partial pooling for grouped data

Foundations of Regression and Predictive Modeling

James Scott | Abhra Sarkar | UT-Austin

# The problem

Industry data is grouped.

- visits handled by physicians,
- users acquired through channels,
- review assignments handled by reviewers,
- customers nested in accounts,
- transactions nested in stores.

And the groups are rarely balanced.

Some groups have thousands of rows. Some have three.

# Why this matters

If we ignore the groups:

*we pretend all observations are independent and comparable.*

If we fit a dummy for every group:

*we estimate noisy effects for tiny groups as if they were just as reliable as large groups.*

Hierarchical models give us a third option.

# The third option

Partial pooling:

> *estimate group-specific effects, but pull weakly estimated groups toward the population average.*

The amount of pull is learned from the data.

Small noisy groups get pulled hard.

Large informative groups get pulled less.

# Opening case

A hospital system studies low-acuity ER visits.

These are not trauma cases. Think sprains, mild infections, simple cuts, routine imaging follow-ups, and other visits likely to end in discharge.

For each visit:

- `DispositionMinutes`: minutes from rooming to discharge/admit decision,
- `Checklist`: whether a structured intake checklist was used,
- `Physician`: attending physician.

Question:

> *Did the checklist shorten time to a disposition decision, after accounting for physician-to-physician differences?*

This is operational. Not a scorecard for clinical quality.

# The data problem

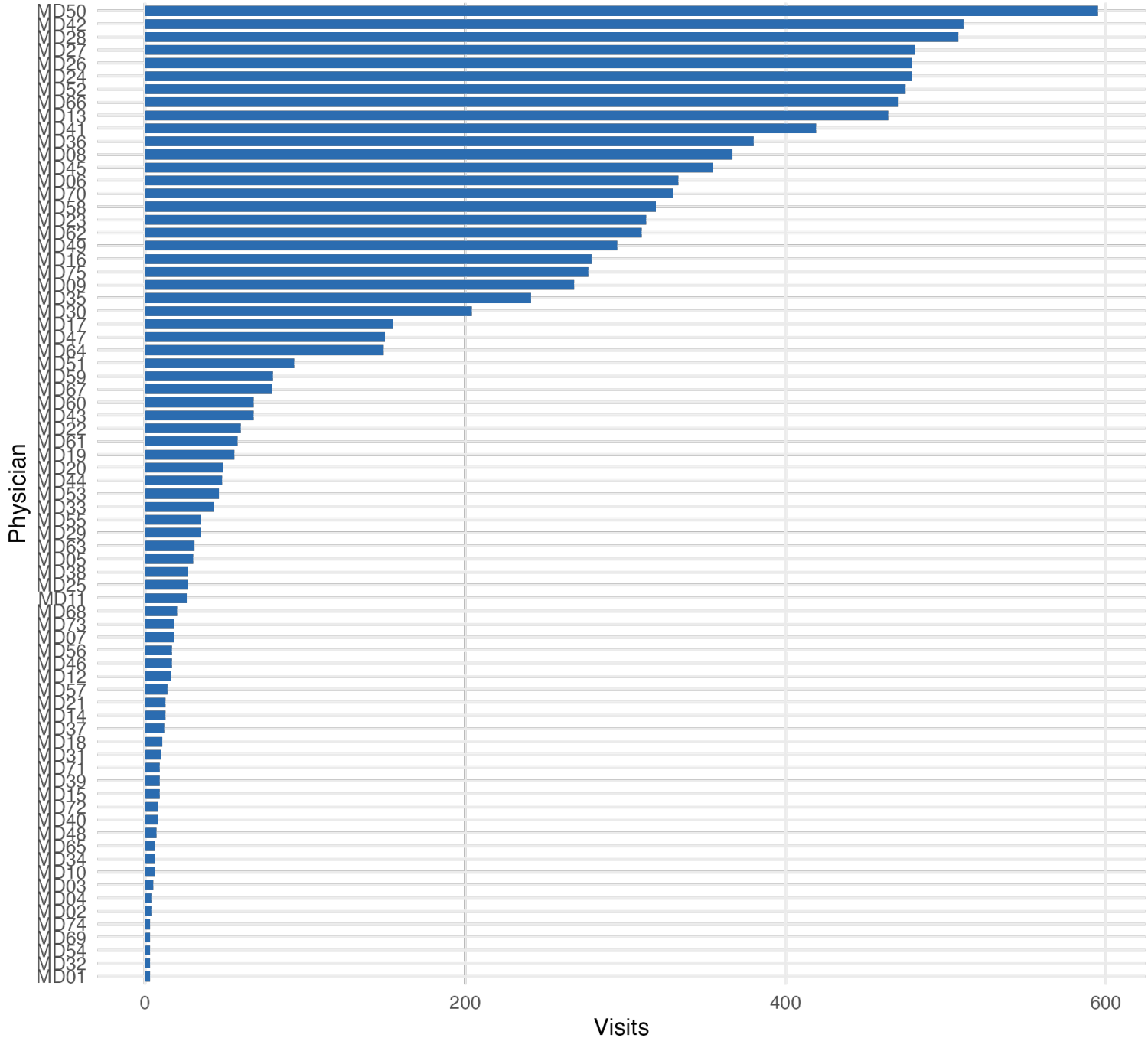
There are 75 physicians and 10,840 low-acuity visits.

But the visit counts are wildly uneven:

- smallest physician count: 3 visits,
- median physician count: 46 visits,
- largest physician count: 595 visits.

## Low-acuity ER visits per physician

Some physicians appear hundreds of times; some appear only three or four times



# First model: ignore physicians

One pooled line:

$$\log(\text{DispositionMinutes}_i) = \beta_0 + \beta_1 \text{Checklist}_i + e_i.$$

This estimates one average checklist effect.

But it assumes the leftover errors are independent after accounting for the checklist.

That is hard to believe: visits handled by the same physician share workflow, shift patterns, habits, and case mix.

# Second model: physician dummies

Add one dummy for each physician:

$$\log(\text{DispositionMinutes}_i) = \beta_0 + \beta_1 \text{Checklist}_i + \gamma_{\text{Physician}_i} + e_i.$$

This allows each physician to have a different baseline.

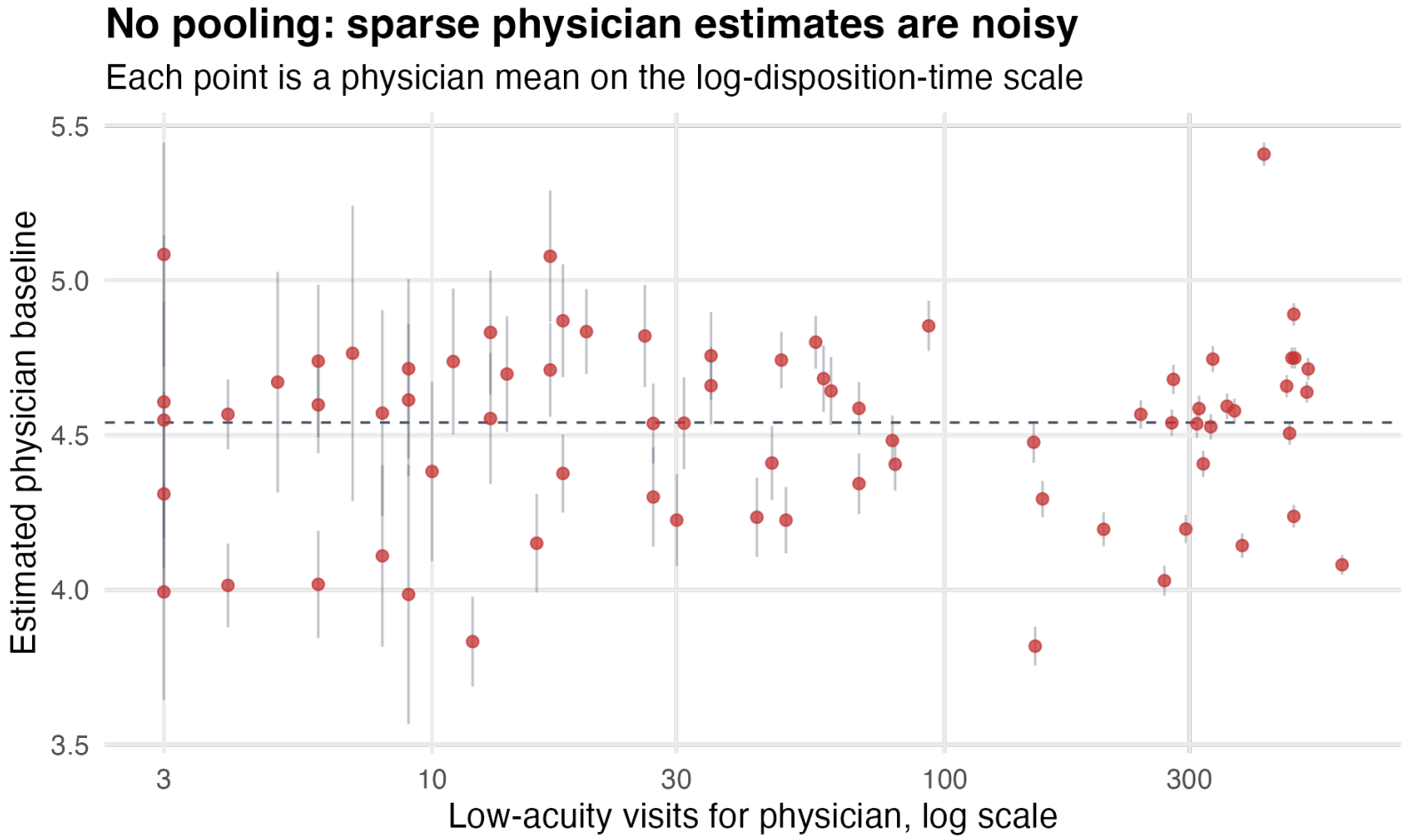
But every physician baseline is estimated separately.

# What happens with `lm()`?

Physicians with three or four visits can get extreme estimated baselines.

Not because we learned something profound about those physicians.

Because three observations are noisy.


No pooling: sparse physician estimates are noisy
Each point is a physician mean on the log-disposition-time scale
Estimated physician baseline
5.5
5.0
4.5
4.0
3.5
3
10
30
100
300
Low-acuity visits for physician, log scale

# The mixed model idea

Let each physician have a baseline offset:

$$\log(\text{DispositionMinutes}_i) = \beta_0 + \beta_1 \text{Checklist}_i + a_{\text{Physician}_i} + e_i.$$

Now add a model for the physician offsets:

$$a_j \sim N(0, \sigma_a^2).$$

That second line is the hierarchy.

# Translation

Each physician gets an intercept.

But the intercepts are assumed to come from a common population of physicians.

So the model asks two questions at once:

- what is the average checklist effect?
- how much do physicians differ in baseline disposition time?

# In R

```
library(rstanarm)

er_fit <- stan_lmer(
  logDispositionMinutes ~ Checklist + (1 | Physician),
  data = er_visits,
  chains = 4,
  iter = 2000,
  seed = 382
)
```

The term `(1 | Physician)` means:

*give each physician a random intercept.*

# What changed?

The checklist effect is about -0.168 on the log scale.

That means checklist visits reach a disposition decision about

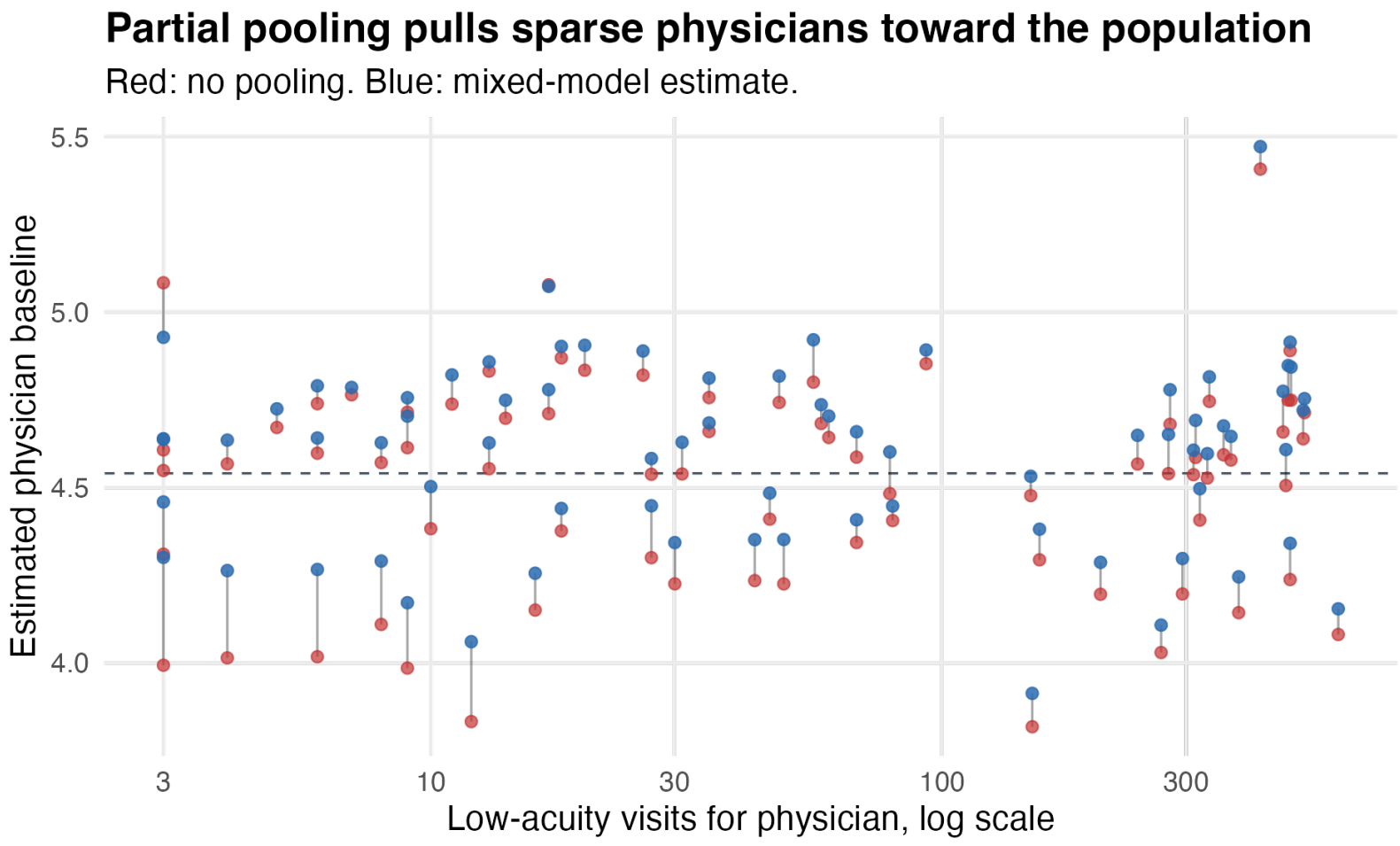
$$e^{-0.168} - 1 \approx -15\%$$

faster, after accounting for physician baselines.

The physician standard deviation is about 0.28 on the log scale.

Physicians really do differ in baseline disposition time.

# Partial pooling



# What the plot says

Red: no pooling.

Blue: partial pooling.

The tiny-physician estimates move most.

Large-physician estimates barely move.

That is borrowing strength.

# Three estimators

No pooling:
*every physician is their own island.*

Complete pooling:
*physicians do not matter.*

Partial pooling:
*physicians matter, but noisy physician estimates are stabilized by the population.*

# Bayesian logic

The model has two levels.

Observation level:

$$y_i \mid a_j \sim N(\beta_0 + \beta_1 x_i + a_j, \sigma^2).$$

Group level:

$$a_j \sim N(0, \sigma_a^2).$$

The group-level distribution is the source of shrinkage.

# Why this is Bayesian

For a small group, its own data are weak.

So the posterior estimate is influenced heavily by the population distribution.

For a large group, its own data are strong.

So the posterior estimate stays closer to the group-specific data.

No magic. Just weighted evidence.

# Random slopes

Sometimes groups differ not only in baseline.

They differ in how a predictor works.

That is a random slope.

# Case 2: document review

A large law firm wants better billing estimates for document review.

Every year, new associates and staff reviewers join matters with very different document loads.

Each row here is one review assignment:

- one reviewer,
- one matter,
- pages reviewed,
- billed hours for document review.

The business question:

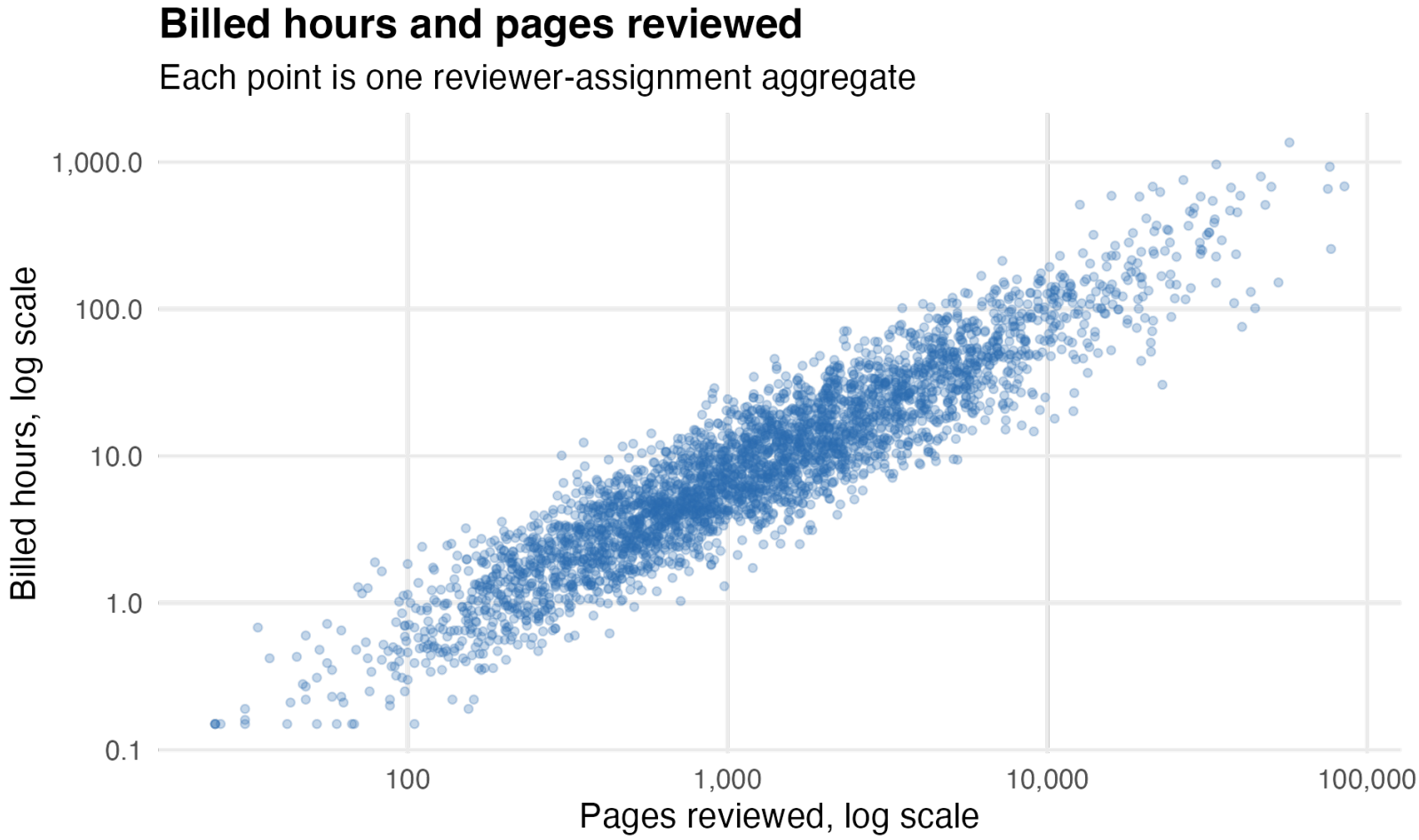
*If the assignment gets larger, how fast do billed hours grow?*

# First look at the scale

Pages reviewed and billed hours both span orders of magnitude.

That is not a detail. It changes the model.

On the original scale, a 500-page assignment and a 50,000-page assignment do not belong in the same straight-line picture.


Billed hours and pages reviewed
Each point is one reviewer-assignment aggregate
1,000.0
100.0
10.0
1.0
0.1
Billed hours, log scale
100
1,000
10,000
100,000
Pages reviewed, log scale

# Why logs?

On the log-log scale, the slope is an exponent:

$$\log(\text{BilledHours}) = \beta_0 + \beta_1 \log(\text{PagesReviewed}) + e.$$

If $\beta_1 = 1$, review time scales roughly linearly with pages.

If $\beta_1 > 1$, review time scales a little faster than linearly.

That is plausible here: large corpora require cross-referencing, issue coding, and connecting documents across the matter.

# Why sparsity is real here

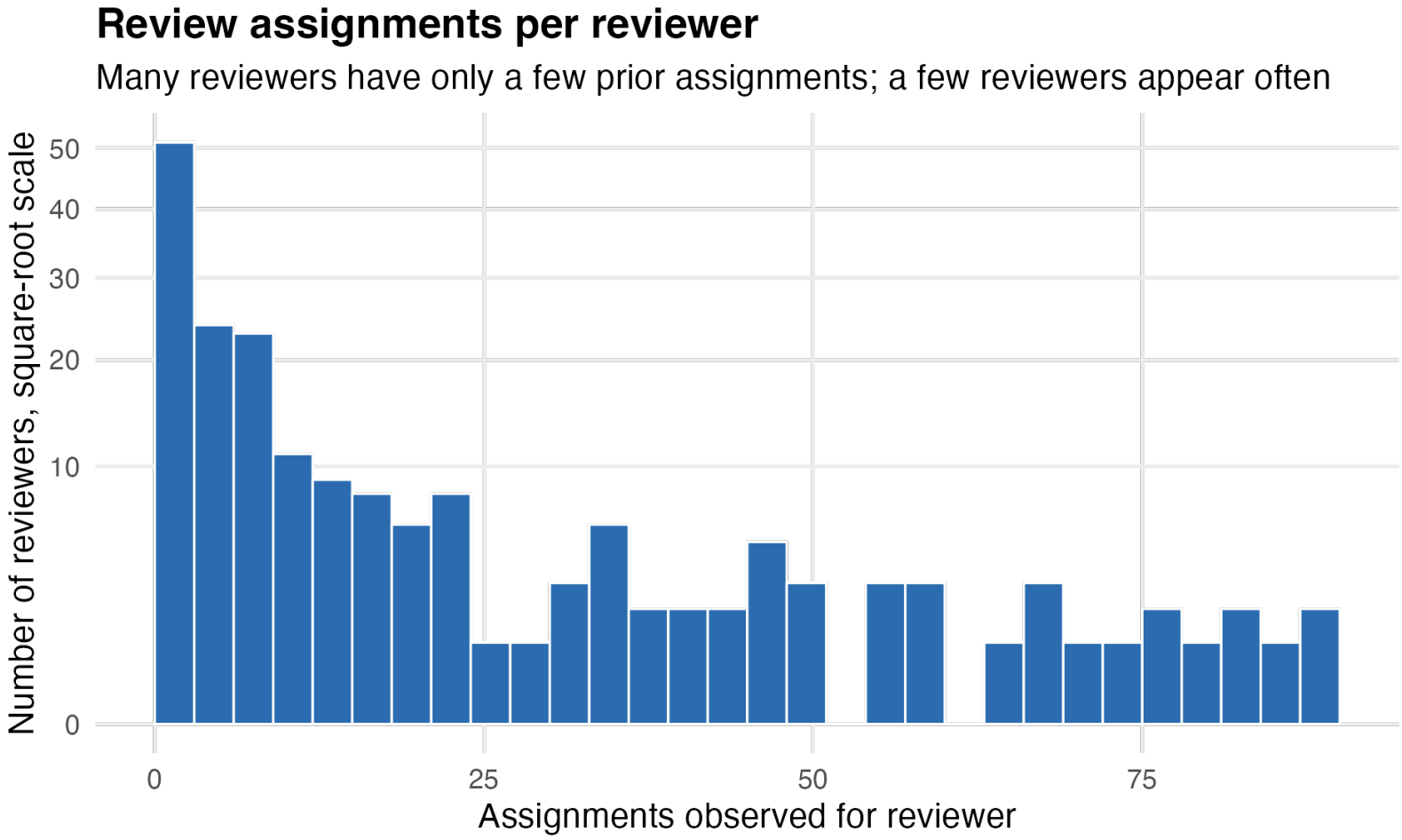
Review histories are not balanced.

- some reviewers do document review constantly,
- some rotate through once,
- new associates may have one or two prior assignments,
- contract reviewers appear for one matter and disappear.

In this sample: 185 reviewers and 3,489 assignments.

The smallest reviewer history is 1 assignment.

The largest is 90.


Review assignments per reviewer
Many reviewers have only a few prior assignments; a few reviewers appear often
Number of reviewers, square-root scale
50
40
30
20
10
0
0
25
50
75
Assignments observed for reviewer

# The question

Outcome:

- `BilledHours`.

Numerical predictor:

- `PagesReviewed`.

Group:

- `ReviewerID`.

More carefully:

*Does the page-to-hours exponent vary by reviewer?*

It should.

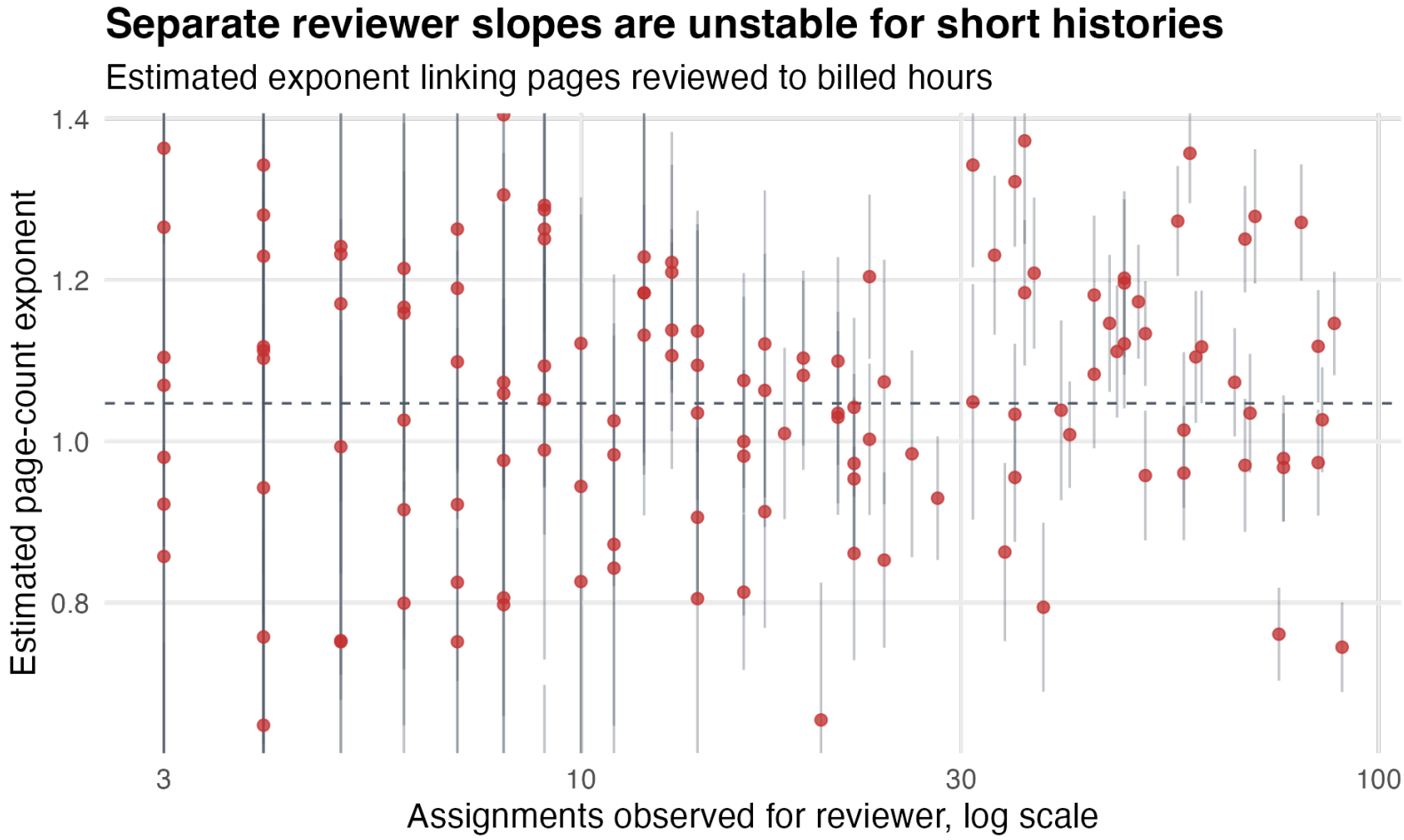
Some reviewers get faster as an assignment gets large. Some slow down when the matter gets complicated.

# What goes wrong separately?

Fit a separate page-count exponent for every reviewer.

Reviewers with two or three assignments can get absurd slopes.

Those slopes are mostly noise.


Separate reviewer slopes are unstable for short histories
Estimated exponent linking pages reviewed to billed hours
Estimated page-count exponent
1.4
1.2
1.0
0.8
3
10
30
100
Assignments observed for reviewer, log scale

# Random slope model

We let both pieces vary by reviewer:

$\log(\text{BilledHours}_{ij}) = \beta_0 + \beta_1 \log(\text{Pages}_{ij}) + \text{document and experience cont}$

Reviewer `j` gets:

- a baseline offset $a_j$,
- a page-count slope offset $b_j$.

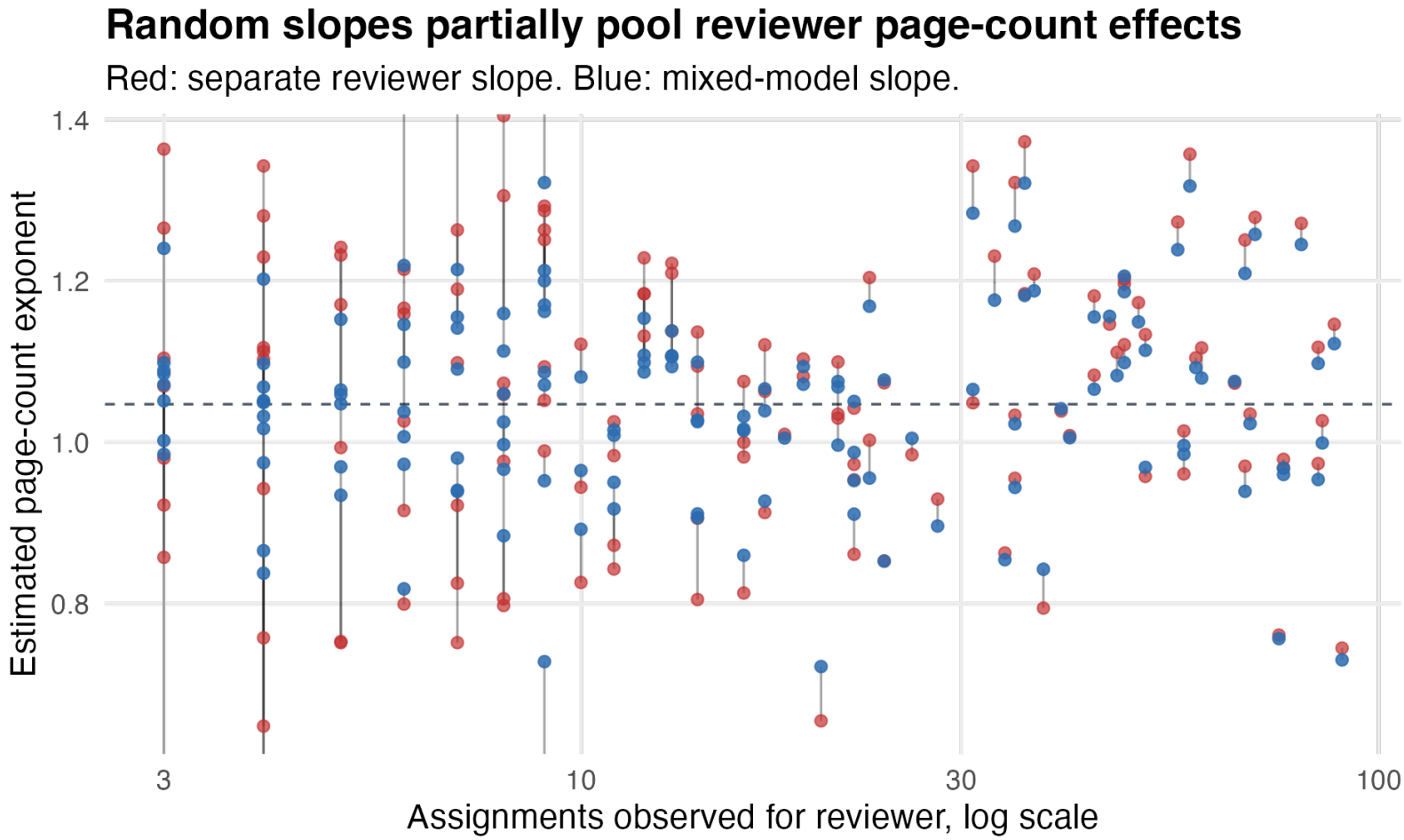
# In R

```
legal_fit <- stan_lmer(
  logBilledHours ~ logPagesCentered + DocumentType +
    ExperienceLevel +
    (1 + logPagesCentered | ReviewerID),
  data = legal_review,
  chains = 4,
  iter = 2000,
  seed = 382
)
```

The term `(1 + logPagesCentered | ReviewerID)` means:

*random reviewer baselines and random reviewer page-count slopes.*

# Partial pooling of slopes



# Reading the model

The average page-count exponent is about `1.05`.

Roughly:

*10% more pages predicts about 10.5% more billed hours.*

That is slightly superlinear scaling.

The reviewer-level slope standard deviation is about `0.14`.

So the page-count penalty really varies across reviewers.

# Case 3: Spredfast

Now the same random-slope idea, but with a count outcome.

Spredfast, in its Austin glory days, sold social media management software to brands, agencies, and media companies.

Customers used it to coordinate publishing, approvals, engagement, analytics, and brand governance across social channels.

Each row here is one customer-month.

The business question:

> *As an account adds active seats, how much does support burden grow?*

# Why sparsity is real here

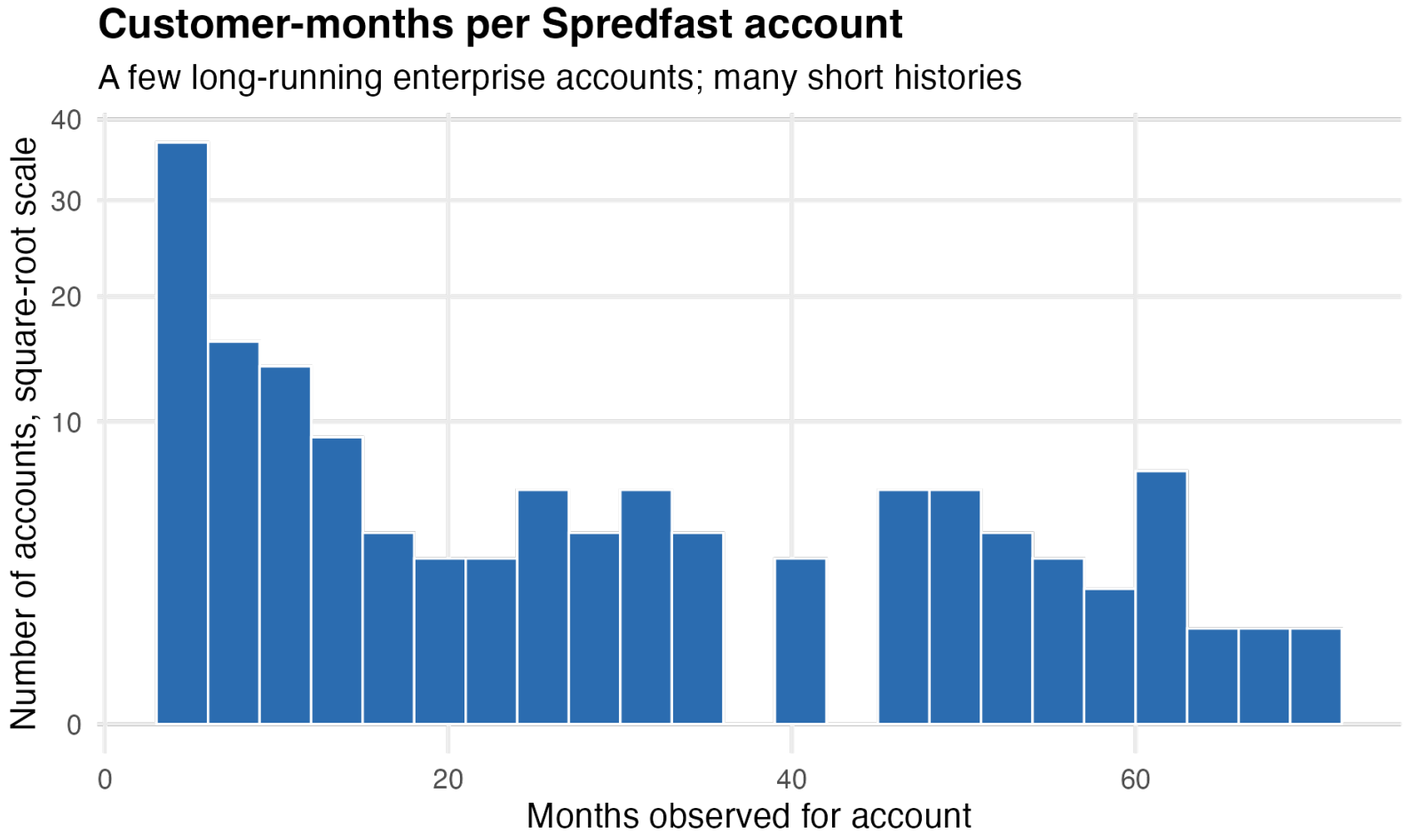
This is not a balanced experiment.

It is a SaaS customer panel.

- new customers enter,
- pilots end,
- accounts churn,
- enterprise customers stay for years,
- a few huge customers dominate the queue.

Some accounts have 70 months of history.

Some have 3.


Customer-months per Spredfast account
A few long-running enterprise accounts; many short histories
Number of accounts, square-root scale
40
30
20
10
0
0
20
40
60
Months observed for account

# The question

Outcome:

- `MonthlyTickets`: support tickets from the account that month.

Numerical predictor:

- `ActiveSeats`: active seats in the account-month.

Group:

- `AccountID`.

More carefully:

*Does the seat-to-ticket slope vary by account?*

Of course it does.

Some customers are self-sufficient. Some customers turn every rollout into a support event.

# What goes wrong separately?

Fit a separate seat-to-ticket slope for every account.

Small-history accounts can look weirdly quiet or wildly support-heavy.

Those slopes are mostly noise.


Separate account slopes are unstable for short histories
Estimated log-seat slope for monthly support tickets
Estimated seat-to-ticket slope
4
0
-4
3
10
30
Customer-months observed, log scale

# Count outcome

Tickets are counts.

So use a Poisson GLMM:

$$\text{Tickets}_{ij} \sim \text{Poisson}(\lambda_{ij}).$$

Then model the log mean:

$$\log(\lambda_{ij}) = \beta_0 + \beta_1 \log(\text{Seats}_{ij}) + a_j + b_j \log(\text{Seats}_{ij}) + \text{controls}.$$

Now account `j` gets its own baseline and its own seat slope.

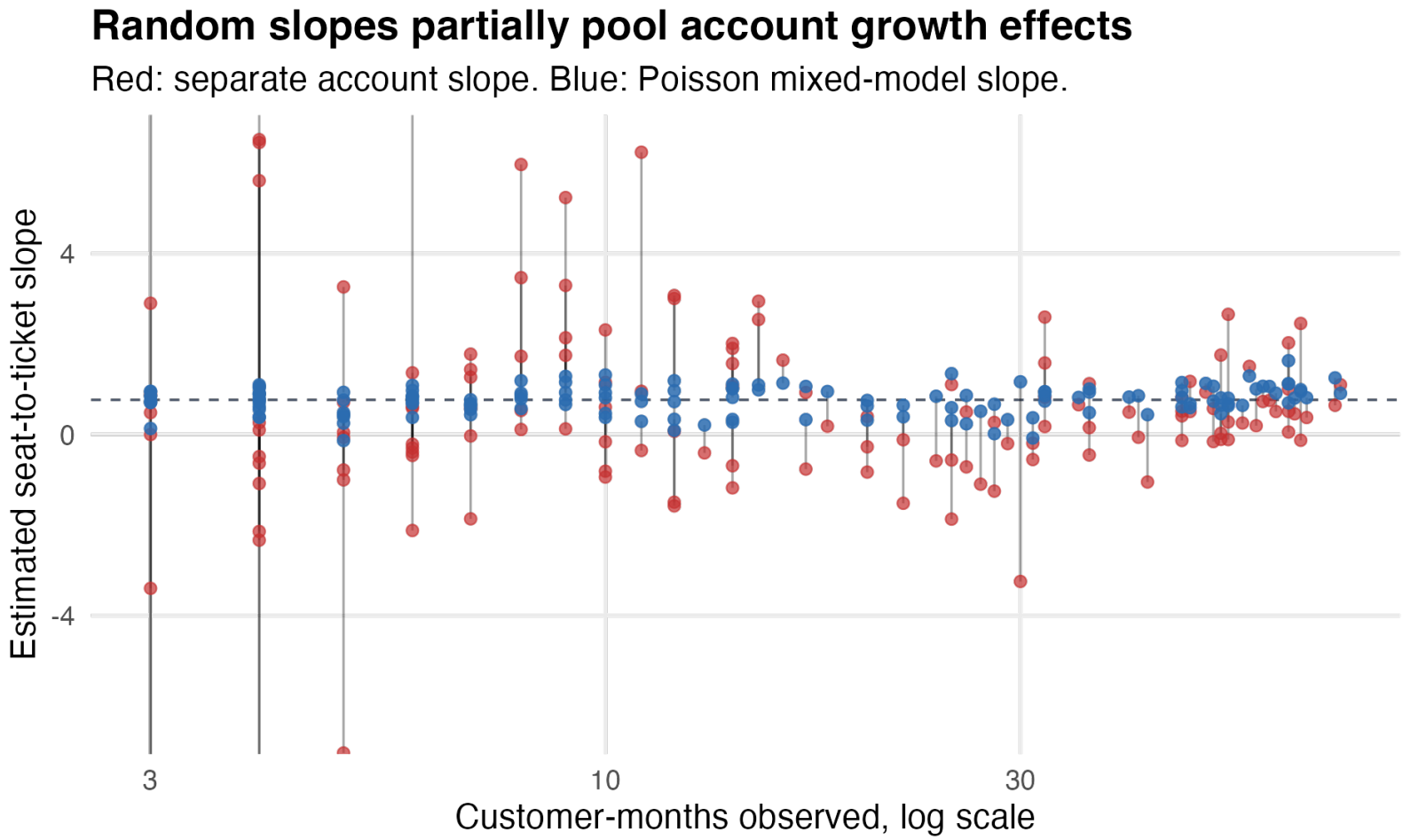
# In R

```
spredfast_fit <- stan_glmer(
  MonthlyTickets ~ logActiveSeatsCentered + PlanTier +
    AccountType + ConnectedProfiles10 + MonthsSinceLaunch12
    (1 + logActiveSeatsCentered | AccountID),
  data = spredfast,
  family = poisson(link = "log"),
  chains = 4,
  iter = 2000,
  seed = 382
)
```

The term `(1 + logActiveSeatsCentered | AccountID)` means:

*random account baselines and random account seat slopes.*

# Partial pooling of slopes



# Reading the model

The average log-seat slope is about `0.76`.

Roughly:

> *a 1% increase in active seats predicts about a 0.76% increase in monthly tickets.*

The account-level slope standard deviation is about `0.45`.

So the support burden of growth really varies across accounts.

# But check the Poisson assumption

For a Poisson model:

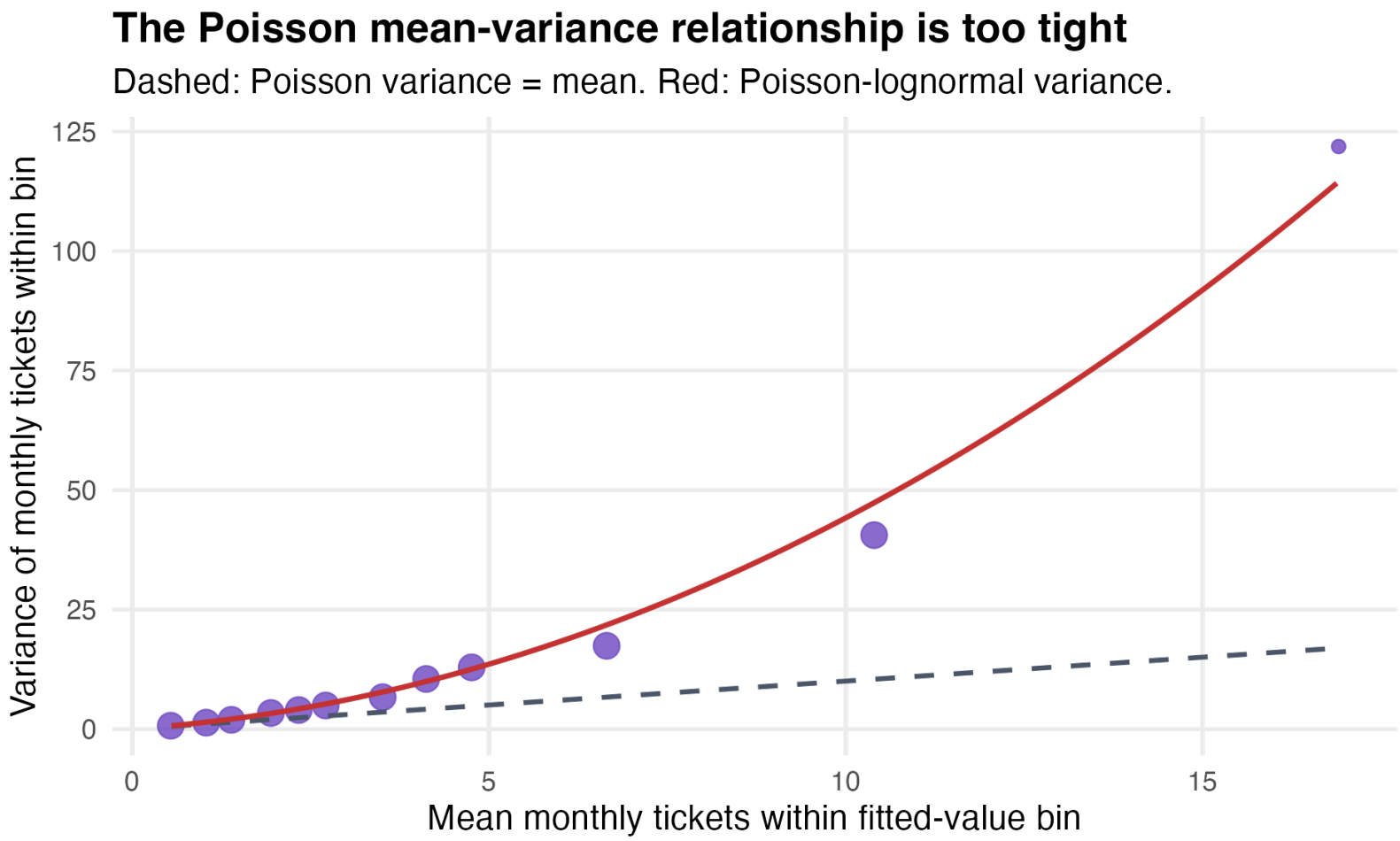
$$\text{variance} = \text{mean}.$$

That is a strong promise.

Support tickets often break it.

Some months are just weird: product incidents, platform API changes, major launches, new workflows.

# Mean-variance check



# Overdispersion

The fitted Poisson model has a Pearson overdispersion ratio around `2.15`.

The data are more variable than a plain Poisson model expects.

One simple fix:

*add an observation-level random effect.*

# Poisson-lognormal model

Add one random intercept per row:

$$\log(\lambda_i) = \text{linear predictor}_i + u_i, \qquad u_i \sim N(0, \sigma_u^2).$$

That is a Poisson-lognormal model.

# In R

```
spredfast_olre <- stan_glmer(
  MonthlyTickets ~ logActiveSeatsCentered + PlanTier +
    AccountType + ConnectedProfiles10 + MonthsSinceLaunch12
    (1 + logActiveSeatsCentered | AccountID) +
    (1 | ObsID),
  data = spredfast,
  family = poisson(link = "log")
)
```

The `(1 | ObsID)` term gives the Poisson model extra row-level variation.

# Why industry uses these models

Borrowing strength:
*sparse groups get stabilized by the population.*

Group dependencies:
*observations inside the same group are not treated as independent clones.*

Unbalanced data:
*different group sizes are not a problem.*

Generalizability:
*we estimate how effects vary across the kinds of groups we will see again.*

# The practical lesson

When there are many groups and uneven data:

- pooled regression is too blunt,
- dummy-variable regression is too noisy,
- mixed models are often the useful middle.

The slogan:

*estimate the group differences, but do not trust tiny groups too much.*